

Universidad de La Salle

Ciencias de Datos

Actividad 7: Sistema CRUD GAD-7

Lenguaje de Programación II

Diunis Pérez

dperez37 | rama: dev_dperez37

24 de mayo de 2026

Repositorio: <https://github.com/dperez18/Actividad7>

Descripción General

Este proyecto implementa un sistema **CRUD** completo sobre la entidad **CuestionarioGAD7**, el cuestionario clínico estandarizado de detección de trastorno de ansiedad generalizada (*Generalized Anxiety Disorder 7*), aplicado a estudiantes universitarios.

El sistema está desarrollado en **Python 3.13** aplicando principios SOLID, Clean Code y PEP 8, con persistencia en formato **JSON** (requerida por la actividad) y soporte adicional en **MySQL Workbench**. La interfaz de usuario es en consola (CLI).

Módulo	Descripción
Modelo	Clases CuestionarioGAD7 y Respuesta : encapsulan los datos, validan entradas (valor 0–3 por ítem) y calculan puntaje y nivel de severidad.
Repositorio DAO	Interfaz IRepository[T] (ABC) con 5 métodos CRUD. Implementaciones concretas en JSON y MySQL.
Servicio	GAD7Service : orquesta la lógica de negocio y valida completitud del cuestionario antes de persistir.
Vista	MenuPrincipal : interfaz CLI con menú numerado y validaciones inmediatas.
Excepciones	Jerarquía AppError → EntidadNoEncontradaError , CuestionarioIncompletoError , ValorFueraDeRangoError .

Análisis y Diseño

Antes de escribir código se identificaron los conceptos de la unidad y cómo aplicarlos al problema de gestión de cuestionarios GAD-7:

Concepto de la unidad	Decisión de diseño
Patrón MVC	Se separó presentación (MenuPrincipal), lógica de negocio (GAD7Service) y datos (CuestionarioGAD7).
Patrón DAO/Repository	Se definió IRepository[T] como interfaz abstracta. El servicio nunca accede directamente a archivos ni a la BD.
Principios SOLID	Cada clase tiene una responsabilidad única. El repositorio es intercambiable sin modificar el servicio.
Persistencia JSON	Requerida por la actividad. El archivo cuestionarios_gad7.json se crea automáticamente si no existe.
Persistencia MySQL	Añadida como mejora. Permite consultar los datos desde MySQL Workbench con SQL estándar sin cambiar ninguna capa del sistema.

Estructura del Proyecto

```

Actividad7/
|-- main.py                                <- Punto de entrada (composition
    root)
|-- requirements.txt
|-- app/
|   |-- exceptions.py                    <- Jerarquia de excepciones de
    dominio
|   |-- interfaces/
|   |   '-- i_repository.py             <- Contrato ABC con 5 metodos
    CRUD
|   |-- models/
|   |   |-- cuestionario_gad7.py        <- Entidad principal (dataclass)
|   |   |-- respuesta.py               <- Entidad Respuesta (dataclass)
|   |   '-- estado_riesgo.py           <- Enum: NORMAL / LEVE / MODERADO
    / SEVERO
|   |-- repositories/
|   |   |-- gad7_repository_json.py     <- Implementacion DAO -> JSON
|   |   '-- gad7_repository_mysql.py    <- Implementacion DAO -> MySQL
    Workbench
|   |-- services/
|   |   '-- gad7_service.py             <- Logica de negocio (CRUD)
|   '-- ui/
|       |-- menu.py                    <- CLI interactivo
|       '-- mensajes.py                <- Constantes de texto
|-- data/
|   '-- cuestionarios_gad7.json         <- Almacenamiento persistente
    JSON
'-- tests/
    '-- test_gad7.py                    <- 29 pruebas unitarias con
        pytest

```

Patrones de Diseño: MVC y DAO/Repository

Patrón MVC (Model – View – Controller)

Definición formal: MVC divide la aplicación en tres componentes con responsabilidades bien delimitadas. El *Modelo* gestiona los datos y la lógica de dominio sin conocer la presentación; la *Vista* presenta la información sin contener lógica de negocio; el *Controlador/Servicio* recibe las acciones del usuario y las delega al Modelo. El objetivo es lograr bajo acoplamiento y alta cohesión entre las capas. (*Gamma et al., “Design Patterns”, 1994.*)

Rol	Clase / Archivo	Responsabilidad
Model	CuestionarioGAD7 models/cuestionario_gad7.py	Valida datos, calcula puntaje y clasifica severidad. No conoce la vista.
View	MenuPrincipal ui/menu.py	Muestra menús y lee input. No contiene lógica de negocio.
Service	GAD7Service services/gad7_service.py	Orquesta el CRUD, valida y delega al repositorio.

Patrón DAO / Repository

Definición formal: El patrón *Data Access Object* (DAO) define una interfaz que abstrae el acceso a una fuente de datos. Toda la lógica de persistencia queda en el objeto concreto; el resto del sistema depende únicamente del contrato abstracto, sin importar si los datos están en un archivo o en una BD. (Fowler, “*Patterns of Enterprise Application Architecture*”, 2002.)

Componente	Clase	Rol
Contrato abstracto	IRepository[T]	ABC genérica con 5 métodos: crear, listar, buscar_por_codigo, actualizar, eliminar.
Implementación JSON	GAD7RepositoryJSON	Persiste con json.load/json.dump. Crea el archivo automáticamente si no existe.
Implementación MySQL	GAD7RepositoryMySQL	Persiste en MySQL Workbench. Crea la BD y tabla automáticamente.
Consumidor	GAD7Service	Recibe IRepository por inyección de dependencias. Nunca instancia el repositorio directamente.

Principios SOLID

Principio	Definición	Evidencia en el proyecto
S — Single Responsibility	Cada clase tiene una única razón para cambiar.	CuestionarioGAD7 solo modela datos; GAD7RepositoryJSON solo persiste; MenuPrincipal solo presenta.
O — Open/Closed	Abierto para extensión, cerrado para modificación.	Para agregar otro motor de BD basta crear una nueva clase que implemente IRepository, sin tocar el servicio ni los modelos.
L — Liskov Substitution	Las subclases sustituyen a su base sin romper el comportamiento.	Los tests usan MagicMock() como sustituto del repositorio real y el servicio funciona exactamente igual.
I — Interface Segregation	Interfaces pequeñas y específicas.	IRepository define solo los 5 métodos del dominio GAD-7, sin métodos de otras entidades del sistema.
D — Dependency Inversion	Depender de abstracciones, no de implementaciones concretas.	GAD7Service.__init__(self, repositorio: IRepository[...]). El único lugar con instancias concretas es main.py.

Sistema GAD-7 — CRUD

Clasificación de Severidad

El puntaje total es la suma de las 7 respuestas (valor 0–3 por ítem; máximo 21). La clasificación sigue los umbrales clínicos estándar del GAD-7:

Nivel	Puntaje	Constante en código
Normal	0 – 4	puntaje < UMBRAL_LEVE = 5
Leve	5 – 9	UMBRAL_LEVE = 5
Moderado	10 – 14	UMBRAL_MODERADO = 10
Severo	15 – 21	UMBRAL_SEVERO = 15

Fragmentos de Código Clave

Modelo — clasificación de severidad:

```
@dataclass
class CuestionarioGAD7:
    codigo_estudiante: str
    respuestas: List[Respuesta] = field(default_factory=list)
    id: str = field(default_factory=lambda: str(uuid.uuid4()))
    fecha_aplicacion: datetime = field(default_factory=datetime.now)

    @property
    def puntaje_total(self) -> int:
        return sum(r.valor for r in self.respuestas)

    def clasificar_severidad(self) -> EstadoRiesgo:
        puntaje = self.puntaje_total
        if puntaje >= UMBRAL_SEVERO: return EstadoRiesgo.SEVERO
        if puntaje >= UMBRAL_MODERADO: return EstadoRiesgo.MODERADO
        if puntaje >= UMBRAL_LEVE: return EstadoRiesgo.LEVE
        return EstadoRiesgo.NORMAL
```

Interfaz DAO — contrato genérico:

```
class IRepository(ABC, Generic[T]):
    @abstractmethod
    def crear(self, entidad: T) -> T: ...
    @abstractmethod
    def listar(self) -> List[T]: ...
    @abstractmethod
    def buscar_por_codigo(self, codigo: str) -> Optional[T]: ...
    @abstractmethod
    def actualizar(self, entidad: T) -> T: ...
    @abstractmethod
    def eliminar(self, codigo: str) -> None: ...
```

Servicio — inyección de dependencias:

```
class GAD7Service:
    def __init__(self, repositorio: IRepository[CuestionarioGAD7]) -> None:
        self._repo = repositorio # abstraccion, no JSON ni MySQL directamente

    def crear_cuestionario(self, codigo: str, respuestas: List[Respuesta]):
        cuestionario = CuestionarioGAD7(codigo_estudiante=codigo,
                                          respuestas=respuestas)

        if not cuestionario.validar():
```

```
raise CuestionarioIncompletoError(
    "Debe tener exactamente 7 respuestas.")
return self._repo.crear(cuestionario)
```

Persistencia en MySQL Workbench

Se utilizó MySQL Workbench como base de datos relacional adicional al JSON requerido. La decisión de usar MySQL se basa en que permite consultar los cuestionarios con SQL estándar, visualizar los datos en tablas y escalar el sistema a producción sin cambiar ninguna capa del código.

El repositorio `GAD7RepositoryMySQL` crea automáticamente la base de datos `gad7_db` y la tabla `cuestionarios_gad7` si no existen, lo que facilita la primera ejecución sin configuración manual.

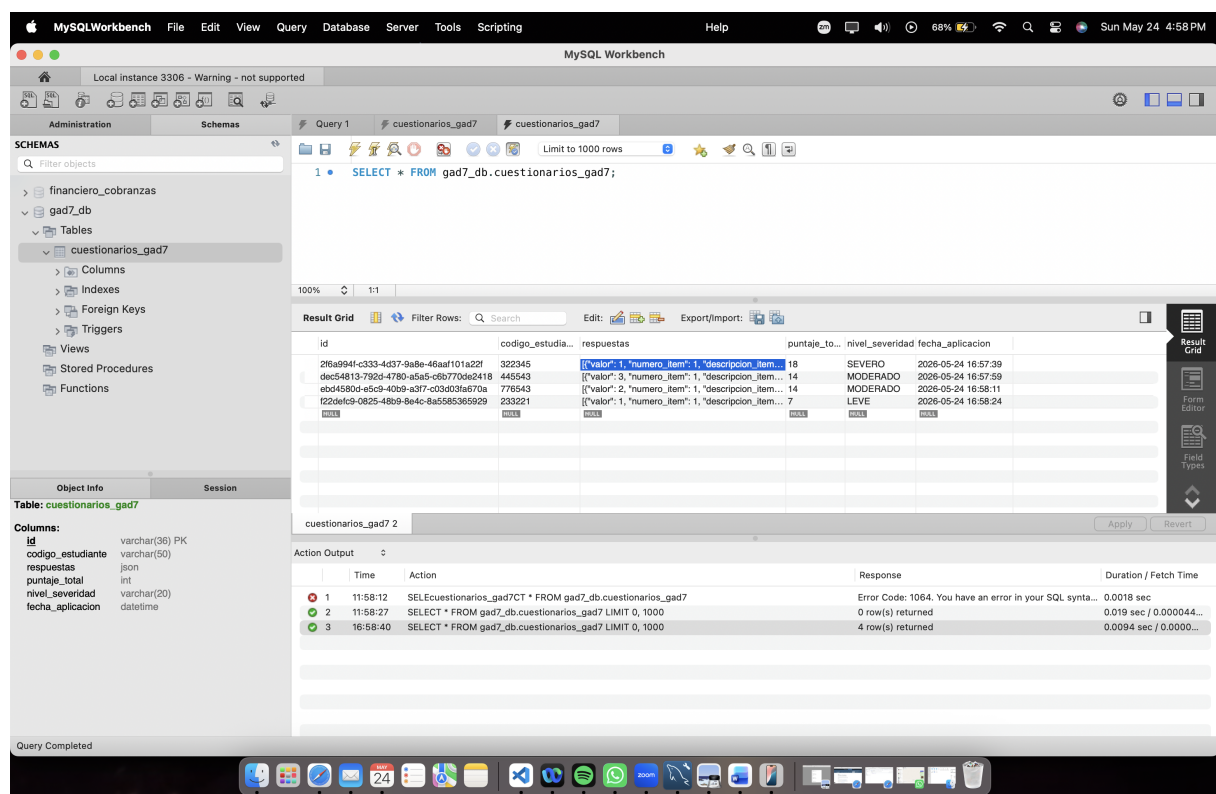


Figura 1: Tabla `cuestionarios_gad7` en MySQL Workbench con registros reales

Pruebas Unitarias

Estrategia

Se diseñaron **29 casos de prueba** con `pytest` distribuidos en 4 clases, cubriendo: validación de entrada, todos los umbrales de severidad como *edge cases*, lógica del servicio

con repositorio simulado (MagicMock) y pruebas de integración del repositorio JSON con archivos temporales (tmp_path).

Resumen de Resultados

Clase de prueba	Tests	Fallos	Errores	Resultado
TestRespuesta	6	0	0	PASSED
TestCuestionarioGAD7	11	0	0	PASSED
TestGAD7Service	7	0	0	PASSED
TestGAD7RepositoryJSONIntegracion	5	0	0	PASSED
TOTAL	29	0	0	OK

Edge Cases Destacados

Test	Qué verifica
test_clasificar_normal_puntaje_4	Puntaje 4 sigue siendo NORMAL
test_clasificar_leve_umbral_exacto_5	Puntaje 5 es el primer LEVE
test_clasificar_moderado_umbral_exacto_10	Puntaje 10 es el primer MODERADO
test_clasificar_severo_umbral_exacto_15	Puntaje 15 es el primer SEVERO
test_clasificar_severo_puntaje_maximo_21	Puntaje máximo sigue siendo SEVERO
test_valor_4_lanza_error	Respuesta con valor 4 lanza ValueError
test_valor_negativo_lanza_error	Respuesta negativa lanza ValueError
test_eliminar_inexistente_lanza_error	Eliminar ID inexistente lanza error

Captura de Ejecución

```
diup@Mac Actividad7 % cd /Users/diup/Desktop/Actividad7
python3.13 main.py
```

SISTEMA GAD-7 – CRUD ANSIEDAD

MENÚ PRINCIPAL

1. Registrar nuevo cuestionario
2. Listar todos los cuestionarios
3. Buscar cuestionario por ID
4. Actualizar respuestas de cuestionario
5. Eliminar cuestionario
6. Evaluar nivel de riesgo
0. Salir

Seleccione una opción: 1

--- REGISTRAR CUESTIONARIO ---

Código del estudiante: 12345

(escriba 'cancelar' para volver al menú)

Escala: [0] Nunca [1] Varios días [2] Más de la mitad [3] Casi siempre

1. Sentirse nervioso/a, ansioso/a o muy alterado/a: nerviosa

Ingrese un número entre 0 y 3.

1. Sentirse nervioso/a, ansioso/a o muy alterado/a: 3

2. No poder dejar de preocuparse o controlar la preocupación: si

Ingrese un número entre 0 y 3.

2. No poder dejar de preocuparse o controlar la preocupación: 3

3. Preocuparse demasiado por cosas diferentes:

Ingrese un número entre 0 y 3.

3. Preocuparse demasiado por cosas diferentes: 3

4. Dificultad para relajarse:

Ingrese un número entre 0 y 3.

4. Dificultad para relajarse: 2

5. Estar tan inquieto/a que es difícil mantenerse sentado/a:

Ingrese un número entre 0 y 3.

5. Estar tan inquieto/a que es difícil mantenerse sentado/a: 1

6. Molestarse o ponerse irritable fácilmente:

Ingrese un número entre 0 y 3.

6. Molestarse o ponerse irritable fácilmente: 2

7. Sentir miedo como si algo terrible fuera a pasar: 3

✓ Cuestionario registrado exitosamente.

ID : 79e7c5ae-f299-4a6b-9043-54137eb9d5e9

Puntaje: 17/21

Nivel : SEVERO

Figura 2: Ejecución de las 29 pruebas unitarias — todas pasando

Ejecución del Programa

```
# Instalar dependencias
pip install -r requirements.txt

# Ejecutar pruebas
python3.13 -m pytest tests/test_gad7.py -v

# Ejecutar el programa (modo JSON)
# En main.py: USE_MYSQL = False
python3.13 main.py

# Ejecutar el programa (modo MySQL Workbench)
# En main.py: USE_MYSQL = True
python3.13 main.py
```

Actividad 7 – Lenguaje de Programación II – Universidad de La Salle – 2026 – dperez37